

6

Nodes, Edges, and Statistical (In)dependence

Welcome to *Part 2* of our book! Congratulations on getting this far!

In this part, we'll dive into the world of causal inference. We'll combine all the knowledge that we've gained so far and start building on top of it. This chapter will introduce us to two powerful concepts – **d-separation** and estimands.

Combining these two concepts with what we've learned so far will equip us with a flexible toolkit to compute causal effects.

Further down the road, we'll discuss **back-door** and **front-door** criteria – two powerful methods to identify causal effects – and introduce a more general notion of Judea Pearl's **do-calculus**. Finally, we'll present **instrumental variables** – a family of techniques broadly applied in econometrics, epidemiology, and social sciences.

After reading this chapter and working through the exercises, you will be able to take a simple dataset and – assuming that the data meets necessary assumptions – compute causal effect estimates yourself.

In this chapter, we will cover the following topics:

- The notion of d-separation
- The notion of an estimand
- Back-door and front-door criteria
- Do-calculus
- Instrumental variables

Ready? Let's go!

You're gonna keep 'em d-separated

In the previous chapter, we learned that colliders have a **unique conditional independence pattern** that sets them apart from chains and forks. The idea of d -separation builds on these properties. In general, we say that two nodes in a **directed acyclic graph (DAG)** G are d -separated when *all paths* between them are *blocked*. When is a path between two nodes blocked?

A simple answer is when there's a *collider* on a path between them or if there's a *fork* or a *chain* that contains another variable that we control for (or a descendant of such a variable).

Let's formalize this definition and make it a little bit more general at the same time. Instead of talking about blocking a path between two nodes with another node, we will talk about paths between *sets of nodes* blocked by another *set of nodes*. We will denote sets of nodes with capital cursive script letters, $\mathcal{X}$, $\mathcal{Y}$, and $\mathcal{Z}$.

Thinking in terms of sets of nodes rather than single nodes is useful when we work with scenarios with multiple predictors and/or multiple confounders. If you prefer thinking in terms of single nodes X , Y , and Z , imagine these nodes as a single-element set, and the same definition will work for you!

Let's go!

For any three disjoint sets of nodes $\mathcal{X}$, $\mathcal{Y}$, and $\mathcal{Z}$, a path between $\mathcal{X}$ and $\mathcal{Y}$ is blocked by $\mathcal{Z}$ in the following scenarios:

- If there's a fork, $i \leftarrow j \rightarrow k$, or a chain, $i \rightarrow j \rightarrow k$, in this path such that the middle node is $j \in \mathcal{Z}$
- If there's a collider, $i \rightarrow j \leftarrow k$, on this path such that neither j nor any of its descendants belong to $\mathcal{Z}$ (Pearl, 2009)

In other words, if there's a chain or fork between $\mathcal{X}$ and $\mathcal{Y}$, we need to control for the middle node to close the path between $\mathcal{X}$ and $\mathcal{Y}$. If there's a collider between $\mathcal{X}$ and $\mathcal{Y}$, we should leave it uncontrolled altogether with all its descendants. That's it!

Information flow in a graph

Note, that sometimes, when talking about d -separation, we might refer to *information flow in a graph*. An important thing to remember is that this information flow is *non-directional*. This lack of directionality is closely related to the notions of correlation and confounding. D -separation allows us to control the information flow in the graph. We'll learn more about this topic in the next section.

Now, let's do some practice!

Practice makes perfect – d-separation

When learning a new concept, a good intellectual understanding is often necessary, but it's practicing that makes us retain knowledge in the long term. To strengthen our understanding of d -separation, let's play a game. We'll call it *Keep 'em d-separated*.

I'll generate five graphs of increasing complexity for you and ask you to indicate which nodes we need to observe (or control for) to d -separate X and Y . In other words, your task is to decide which nodes should become the members of the set $\mathcal{Z}$ in order for X and Y to be d -separated. You will find the correct answers at the end of this section.

Figure 6.1 presents our two first examples:

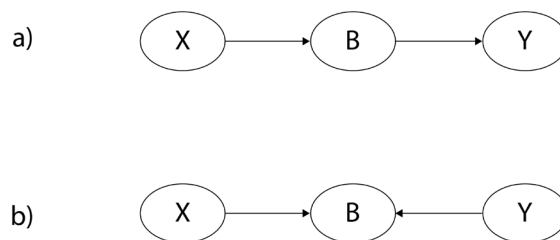


Figure 6.1 – The first two DAGs in the Keep 'em d-separated game

Which nodes of DAGs **a** and **b** from Figure 6.1 should be observed in order to make the X and Y nodes d -separated? I encourage you to write your answers down on a piece of paper and then compare them with the answers at the end of the chapter.

OK, let's add some more complexity! Figure 6.2 presents the third DAG in our game:

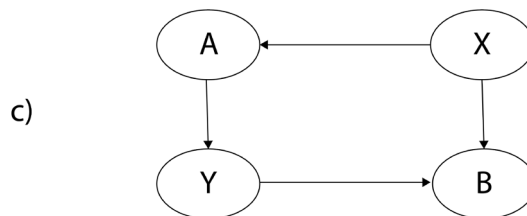


Figure 6.2 – DAG number 3 in our Keep 'em d-separated game

Got your answers?

Great, let's make it more fun! Figure 6.3 presents the fourth example in *Keep 'em d-separated*:



Figure 6.3 – DAG number 4 in our Keep 'em d-separated game

DAGs similar to DAG **d** in Figure 6.3 are often more challenging, but I am sure you'll find the right answer!

OK, now, it's time for something more advanced! Figure 6.4 presents the last example in our game:

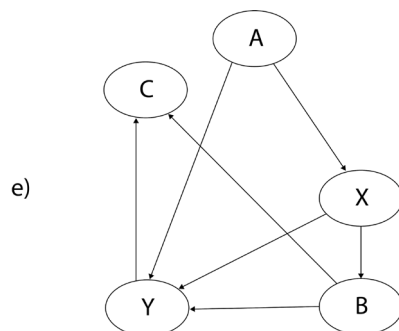


Figure 6.4 – The final DAG in the Keep 'em d-separated game

Do you have your answers ready?

If you found some of the DAGs challenging, that's completely natural.

I am sure that, with practice, you'll feel more confident about them! You can find more similar DAG games on the pgmpy web page (<https://bit.ly/CausalGames>).

It's time to reveal the answers now!

Figure 6.5 contains all five DAGs for your reference.

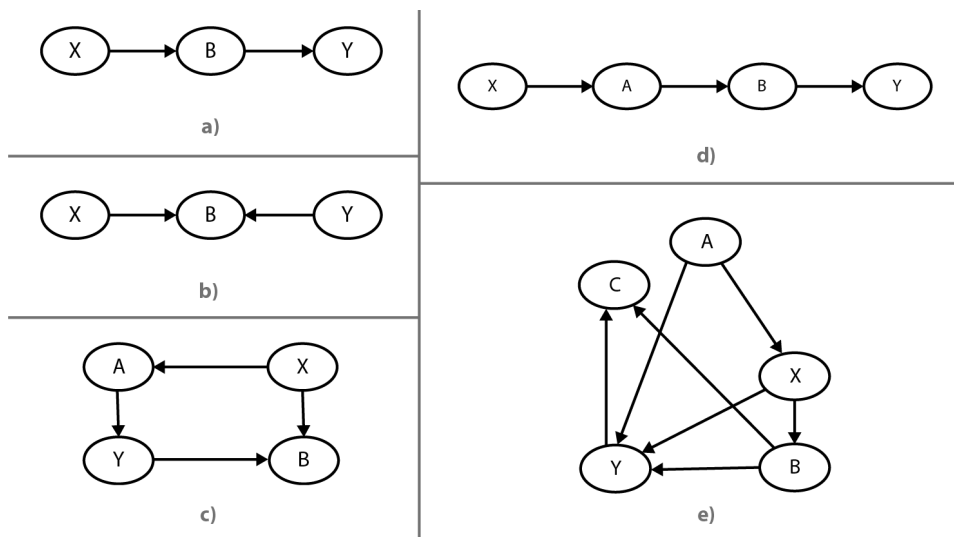


Figure 6.5 – All DAGs from the Keep 'em d-separated game

Let's start with DAG **a**.

DAG **a** is a chain. To block the path between X and Y , we need to control for B .

DAG **b** is a collider. This means that the path is already blocked by the middle node, and we don't need to do anything.

In DAG **c**, we can find two paths connecting X and Y :

- $X \rightarrow A \rightarrow Y$
- $X \rightarrow B \leftarrow Y$

The first path is a chain. In order to block it, we need to control for A . The second path is a collider, and we don't need to do anything to block it, as it's already blocked.

Note that you can view DAG **c** as a combination of DAGs **a** and **b**.

Let's get another one. We can see that in DAG **d**, there are two overlapping structures:

- A collider, $X \rightarrow A \leftarrow B$
- A fork, $A \leftarrow B \rightarrow Y$

There are several options for how to approach DAG **d**:

- The simplest one is to do nothing (the collider blocks the path).
- We can also control for B – there's no benefit to this, but it also does not harm us (except that it increases model complexity).
- Finally, we can control for A (which will open the path) and for B , which will close the path.

If we're only interested in estimating the effect of X on Y , most of the time, we would likely choose the first answer as the simplest one.

Okay, let's jump to our last friend, DAG **e**.

Let's unpack it. There are essentially two paths from X to Y :

- $X \rightarrow A \rightarrow Y$
- $X \rightarrow B \leftarrow C \rightarrow Y$

We can see right away that the first path represents a chain. Controlling for A closes this path. The second path contains a collider, so not controlling for anything keeps the path closed. Another solution is to block for A , B , and C (note how controlling for C closes the path that we opened by controlling for B).

With this exercise, we conclude the first section of *Chapter 6*. Let's review what we've learned.

We introduced the notion of *d*-separation. We saw that *d*-separation is all about blocking paths between (sets of) nodes in a DAG. Paths in a graph can be blocked by using the fundamental logic of the tools that we already have in our toolbox, the three basic conditional independence structures – chains, forks, and colliders (or immoralities).

The knowledge of *d*-separation effectively enables us to build *estimands* – an essential step in the *four-step causal inference process*. Let's learn more!

Estimand first!

In this section, we're going to introduce the notion of an *estimand* – an essential building block in the causal inference process.

We live in a world of estimators

In statistical inference and machine learning, we often talk about *estimates* and *estimators*. Estimates are basically our best guesses regarding some quantities of interest given (finite) data. Estimators are computational devices or procedures that allow us to map between a given (finite) data sample and an estimate of interest.

Let's imagine you just got a new job. You're interested in estimating how much time you'll need to get from your home to your new office. You decide to record your commute times over 5 days. The data you obtain looks like this:

[22.1, 23.7, 25.2, 20.0, 21.8]

One thing you can do is to take the arithmetic average of these numbers, which will give you the so-called sample mean - your estimate of the true average commute time. You might feel that this is not enough for you and you'd prefer to fit a distribution to this data rather than compute a point-wise estimate (this approach is commonly used in Bayesian statistics).

The arithmetic mean of our data, as well as the parameters of the distribution that we might have decided to fit to this data, are *estimates*. A computational device that we use to obtain estimates is an *estimator*.

Estimators might be as simple as computing the arithmetic mean and as complex as a 550 billion-parameter language model. Linear regression is an estimator, and so is a neural network or a random forest model.

So, what is an estimand?

The basic answer is that an *estimand* is a quantity that we're interested in estimating. If an estimator is the *how*, an estimand is the *what*.

Let's get some context to understand the significance of estimands in causal inference.

In *Chapter 1*, we said that confounding is a causal concept, and to discuss it, we need to go beyond the realm of sheer data. In *Chapter 4*, we learned that *graphical causal models* can encode the causal structure of a system that we're modeling. In the first section of this chapter, we learned that an important property of *d*-separation is that it allows us to control the flow of information in a graph.

Let's try to stretch our thinking about confounding a bit for a minute. What does it mean that a relationship between two variables is confounded?

Can you recall the example from the first chapter (ice cream, drownings, and temperature, presented in *Figure 1.1*)? You can use *Figure 6.6* as a quick refresher:

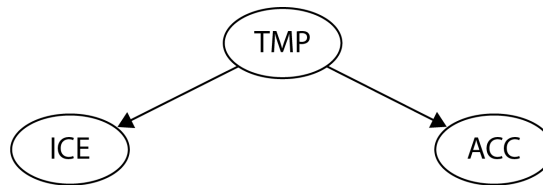


Figure 6.6 – A graphical representation of the problem from Chapter 1 (*Figure 1.1*)

In this example, we perceived ice cream sales (*ICE*) and the number of accidents with drownings (*ACC*) as related although, actually, they were not (causally) related. We said that this relationship was *spurious*.

We can think of this type of relationship as a result of the unconstrained (undirected) flow of information in the graph. This is great news because now we have a *d*-separation that can help us constrain this flow and – as a consequence – *deconfound* the relationship between the variables that we're interested in.

Getting back to our example from the first chapter, we could naively choose to define our model as the following:

$$ACC \sim ICE$$

Let's translate this model syntax to mathematical notation.

What we want to estimate is the *causal effect* of *ICE* on *ACC*. In other words, we want to understand what the change would be in *ACC* if we *intervened* on *ICE*. Therefore, the quantity we're interested in is the following (for the sake of simplicity, we will assume that all our variables are discrete):

$$P(ACC = acc | do(ICE = ice))$$

If we used our naïve model, our estimand would look like this:

$$P(ACC = acc | do(ICE = ice)) = P(ACC = acc | ICE = ice)$$

We already know that this is incorrect in our case. We know that the relationship between *ACC* and *ICE* is spurious! To get the correct *estimate* of the causal effect of *ICE* on *ACC*, we need to control for temperature (*TMP*). The correct way to model our problem is, therefore, the following:

$$ACC \sim ICE + TMP$$

This translates to the following:

$$P(ACC = acc | do(ICE = ice)) = \sum_{tmp} P(ACC = acc | ICE = ice, TMP = tmp) P(TMP = tmp)$$

To make it a bit more readable, we can simplify the notation:

$$P(ACC | do(ICE)) = \sum_{tmp} P(ACC | ICE, TMP) P(TMP)$$

This formula is an example of the so-called **causal effect rule**, which states that given a graph, G , and a set of variables, Pa , that are (causal) parents of X , the causal effect of X on Y is given by the following (Pearl, Glymour, and Jewell, 2016):

$$P(Y = y | do(X = x)) = \sum_z P(Y = y | X = x, Pa = z) P(Pa = z)$$

Note that what we're doing in our example is congruent with point 1 in our definition of d -separation – if there's a fork between the two variables, X and Y , we need to control for the middle node in this fork in to order block a path between X and Y .

In our example, *TMP* is the middle node in the fork between *ICE* and *ACC*, and so controlling for it blocks the non-causal path between *ICE* and *ACC*. This is how we obtain a correct *estimand* for our model (sometimes, there might be more than one correct estimand per model).

All that we've done so far in this section has *one essential goal* – to find an estimand that allows us to compute unbiased causal effects from observational data. Although it won't be always possible, it will be *possible* sometimes. And sometimes, it can bring us tremendous benefits.

Let's summarize this section.

In this section, we learned what *estimands* are and how they are different from estimators and *estimates*. We built a correct estimand for our ice cream example from the first chapter and showed how this estimand is related to a more general *causal effect rule*. Finally, we discussed the links between estimands, d -separation, and confounding.

In the following sections, we'll focus on techniques that allow us to obtain causal estimands, given complete or partially complete graphs.

The back-door criterion

The back-door criterion is most likely the best-known technique to find causal estimands given a graph. And the best part is that you already know it!

In this section, we're going to learn how the back-door criterion works. We'll study its logic and learn about its limitations. This knowledge will allow us to find good causal estimands in a broad class of cases. Let's start!

What is the back-door criterion?

The back-door criterion aims at blocking spurious paths between our treatment and outcome nodes. At the same time, we want to make sure that we leave all directed paths unaltered and are careful not to create new spurious paths.

Formally speaking, a set of variables, $\mathcal{Z}$, satisfies the back-door criterion, given a graph G , and a pair of variables, if no node in $\mathcal{Z}$ is a descendant of X , and $\mathcal{Z}$ blocks all the paths between X and Y that contain an arrow into X (Pearl, Glymour, and Jewell, 2016).

In the preceding definition, $X \rightarrow \dots \rightarrow Y$ means that there is a *directed* path from X to Y . This path might be direct or pass through other nodes.

You can see that when we looked for the estimand in our ice cream example, we did precisely this – we blocked all the paths between *ICE* and *ACC* that contained an arrow into *ICE*. *TMP* is not a descendant of *ICE*, so we also met the second condition. Finally, we haven't opened any new spurious paths (in our very simple graph, there was not even the opportunity to do so).

Back-door and equivalent estimands

Let's consider the graph in *Figure 6.7*:

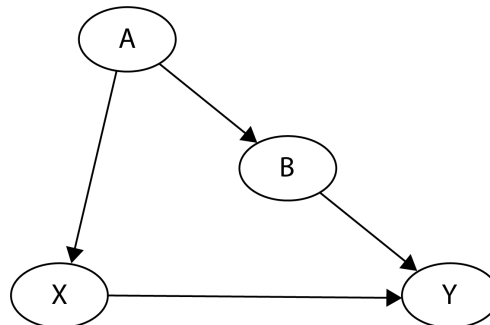


Figure 6.7 – A graph with a confounding pattern

Given the model presented in *Figure 6.7*, which nodes should we control for in order to estimate the causal effect of X on Y ?

According to our definition of the back-door criterion, we need to block all the paths that have an arrow *into* X . We should not control for any descendants of X nor open any new paths. We can fulfill these conditions in three different ways:

- Controlling for A
- Controlling for B
- Controlling for both – A and B

They will all provide us with different but *equivalent* estimands. In particular, the following equality is true for our model (again, assuming discrete variables for simplicity):

$$P(Y = y | do(X = x)) = \sum_a P(Y = y | X = x, A = a)P(A = a) = \sum_b P(Y = y | X = x, B = b)P(B = b)$$

This equality opens a very interesting possibility to us (note that we omitted the third case (controlling for A and B) in the equality for the sake of readability).

If it is sufficient to only control for one of the variables (A or B) to obtain a correct estimand for $X \rightarrow Y$, we can essentially estimate the causal effect of X on Y even if one of the variables remains unobserved!

Equivalent estimands versus equal estimates

Although for certain models we might find two or more equivalent estimands, estimates computed based on these (equivalent) estimands might differ slightly. This is natural in a finite sample size regime. Nonetheless, if your sample size is big enough, the differences should be negligible. Big differences might suggest an erroneous estimand, a lack of model convergence, or errors in the model code.

Let's consider a modified model from *Figure 6.7*, where one of the variables is unobserved. *Figure 6.8* presents this model:

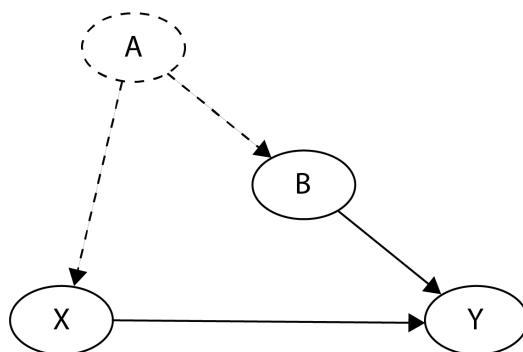


Figure 6.8 – A graph with a confounding pattern and one unobserved variable

The node A and two edges ($A \rightarrow B$ and $A \rightarrow X$) marked with dashed lines are all unobserved, yet we assume that the overall causal structure is known (including the two unobserved edges).

In other words, we don't know anything about A or what the functional form of A 's influence on X or B is. At the same time, we assume that A exists and has no other edges than the one presented in *Figure 6.8*.

A refresher on the do-operator

The *do*-operator informs us that we're working with interventional rather than observational distribution. In certain cases, interventional and observational distributions might be the same. For instance, if your true causal graph has a form of $X \rightarrow Y$, then $P(Y = y | do(X = x)) = P(Y = y | X = x)$, yet whenever confounding appears, we need to adjust for the confounders' effects by controlling for additional variables in the right-hand side of the equation.

Our estimand for the model presented in *Figure 6.8* would be identical to the second estimand for the fully observed model:

$$P(Y = y | do(X = x)) = \sum_b P(Y = y | X = x, B = b) P(B = b)$$

That's powerful! Imagine that recording A is the most expensive part of your data collection process. Now, understanding the back-door criterion, you can essentially just skip recording this variable! How cool is that?

One thing we need to remember is that to keep this estimand valid, we need to be sure that the overall causal structure holds. If we changed the structure a bit by adding a direct edge from A to Y , the preceding estimand would lose its validity.

That said, if we completely removed A and all its edges from the model, our estimand would still hold. Can you explain why? (Check the *Answers* section at the end of the chapter for the correct answer.)

Let's summarize.

In this section, we learned about the back-door criterion and how it can help us build valid causal estimands. We saw that, in some cases, we might be able to build more than one valid estimand for a single model. We also demonstrated that the back-door criterion can be helpful in certain cases of unobserved confounding.

Although the back-door criterion is powerful, it has its limitations.

The front-door criterion

In this section, we're going to discuss the **front-door criterion** – a device that allows us to obtain valid causal estimands in (some) cases where the back-door criterion fails.

Can GPS lead us astray?

In their 2020 study, Louisa Dahmani and Véronique Bohbot from McGill University showed that there's a link between GPS usage and spatial memory decline (Dahmani and Bohbot, 2020). Moreover, the effect is *dose-dependent*, which means that the more you use GPS, the more spatial memory decline you experience.

The authors argue that their results suggest a causal link between GPS usage and spatial memory decline. We already know that something that *looks* connected does not necessarily *have to* be connected in reality.

The authors also know this, so they decided to add a longitudinal component to their design. This means that they observed people over a period of time, and they noticed that those participants who used more GPS had a greater decline in their memory.

Imagine that you decide to discuss this study with your colleague Susan. The time component seems promising to you, but Susan is somehow critical about the results and interpretation and proposes another hypothesis – the link between GPS usage and spatial memory decline is *purely spurious*.

They seem related – argues Susan – because there's a common cause for using GPS and memory decline – **low global motivation** (Pelletier et al., 2007). Susan argues that people with low global motivation are reluctant to learn new things (so they are not interested in remembering new information, including spatial information) and they try to avoid effort (hence, they prefer to use GPS more often, as it allows them to avoid the effortful process of decision-making while driving).

She also claims that low global motivation tends to *expand* – unmotivated people look for solutions that can take the burden of doing stuff from them, and if these solutions work, they use them more often. They are also less and less interested in learning new things with age.

Inspired by Susan's proposition, you search through studies on the effects of GPS on spatial memory, but you cannot find one that would control for global motivation. The situation seems hopeless – if global motivation is a confounder, it's an unobserved one, and so, we cannot use the back-door criterion to deconfound the relationship between GPS usage and spatial memory decline.

Let's encode Susan's model graphically alongside the model containing both hypotheses (motivation and GPS usage). Figure 6.9 presents both models:

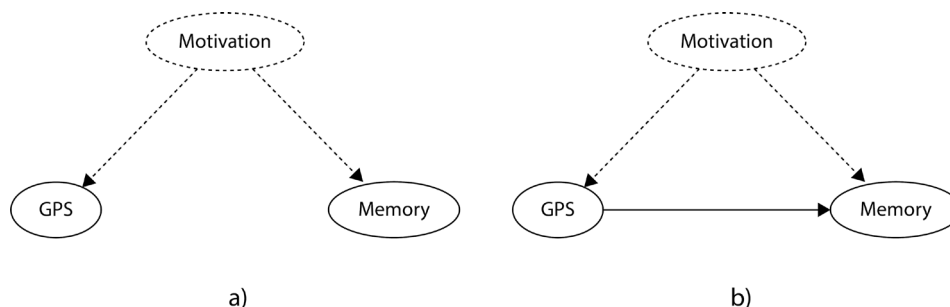


Figure 6.9 – A model presenting Susan's hypothesis (a) and the "full" hypothesis (b)

As we can see, none of the two models can be deconfounded because the confounder is unobserved (dashed lines).

In such a case, the back-door criterion cannot help us, but there's another criterion we could possibly use that relies on the concept of *mediation*. This would require us to find a variable that *mediates* the relationship between GPS usage and memory decline.

Let's look for one!

Mediators and mediation

We can say that the influence of one variable (X) on another (Y) is *mediated* by a third variable, Z (or a set of variables, $\mathcal{Z}$), when at least one path from X to Y goes through Z . We can say that Z *fully mediates* the relationship between X and Y when the only path from X to Y goes through Z . If there are paths from X to Y that do not pass through Z , the mediation is *partial*.

London cabbies and the magic pebble

In their famous 2000 study of London cab drivers, Eleanor A. Maguire and her colleagues from University College London demonstrated that experience as a taxi driver is related to hippocampus's volume (Maguire et al., 2000). The **hippocampus** is a pebble-sized (40-55 mm) brain structure, responsible for creating new memories – in particular, spatial memories (O'Keefe and Nadel, 1978).

London cab drivers need to pass a very restrictive exam checking their spatial knowledge and are not allowed to use any external aids in the process.

The exam is preceded by an extensive training period, which typically takes between 3 and 4 years. The drivers are required to memorize and be able to navigate over “26,000 streets and thousands of points of interest in London” (Griesbauer et al., 2021).

One study (Woollett and Maguire, 2011) showed that drivers who failed this exam did not show an increase in hippocampal volume. At the same time, in those who passed the exam, a systematic increase in hippocampal volume was observed. During the continuing training over a 4-year period, hippocampal volume was associated with an improvement in spatial memory (only in those who were in continuous training).

Let's try to incorporate these results into our model. We'll hypothesize that GPS usage negatively impacts the relative volume of the hippocampus, which in turn impacts spatial memory. *Figure 6.10* presents the updated model:

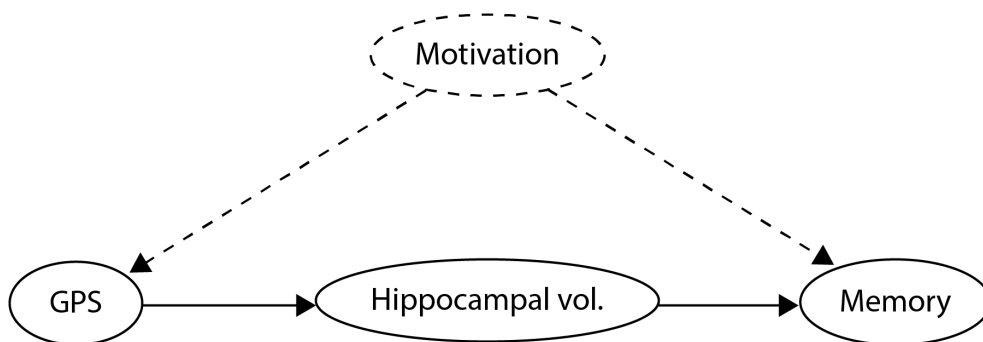


Figure 6.10 – An updated model, including hippocampal volume as a mediator

In our new hypothetical model, we assume that hippocampal volume fully mediates the effects of GPS usage on a decline in spatial memory.

The second important assumption we make is that motivation can only affect *hippocampal volume indirectly through GPS usage*. This assumption is critical in order to make the criterion that we’re going to introduce next – the front-door criterion – useful to us.

If motivation would be able to influence hippocampal volume *directly*, front-door would be of no help. Luckily enough, the assumption that motivation cannot directly change the volume of the hippocampus seems reasonable (though perhaps you could argue against it!).

Opening the front door

The front-door criterion is an example of a divide-and-conquer strategy. It divides a graph into two parts, uses relatively simple rules to determine the causal effects in these sub-parts, and combines them together again.

Let’s see it step by step.

To make the notation more readable, we’ll replace variable names with symbols. *Figure 6.11* presents the graph with updated variable names:

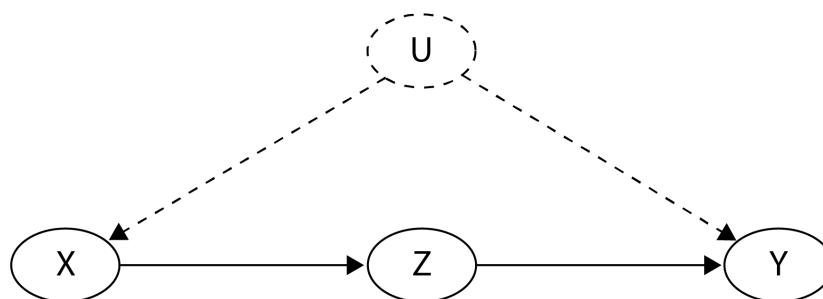


Figure 6.11 – A model with updated variable names

First, let's take a look at the relationship between X and Z . There's one back-door path between them, $X \leftarrow U \rightarrow Y \leftarrow Z$, but it's already blocked. Can you see how?

There's a collider, $U \rightarrow Y \leftarrow Z$, that blocks the flow of the information. Therefore, we can identify the causal effect of X on Z in the following way:

$$P(Z = z | do(X = x)) = P(Z = z | X = x)$$

That's great!

How about the effect of Z on Y ?

There's one open back-door path, $Z \leftarrow X \leftarrow U \rightarrow Y$. There's no collider on this path and U is unobserved, so we cannot control for it. Fortunately, we can control for the other variable, X . A valid estimand of the causal effect of Z on Y is, therefore, the following:

$$P(Y = y | do(Z = z)) = \sum_x P(Y = y | Z = z, X = x) P(X = x)$$

That's pretty awesome! We just blocked the back-door path from Z to Y by simply controlling for X . Now, we're ready to combine both estimands back together:

$$P(Y = y | do(X = x)) = \sum_z P(Y = y | do(Z = z)) P(Z = z | do(X = x))$$

Now, let's drop *do*-operators from the right-hand side by substituting them according to the preceding equalities. This leads us to the following:

$$P(Y = y | do(X = x)) = \sum_z P(Z = z | X = x) \sum_{x'} P(Y = y | X = x', Z = z) P(X = x')$$

This formula is called the **front-door formula** (Pearl et al., 2016) or **front-door adjustment**.

It might look a little discouraging, perhaps because of the cryptic x' in the index of the second sum. We need this unfriendly x' because we're combining two different formulas, and we want to hold X from one formula constant when we're iterating over (the same) X from the other formula.

You can think about it as seeing X from two different angles or – perhaps – as X being in two different places at the same time (I guess in sci-fi movies they usually call it *bilocation*).

Three simple steps toward the front door

In general, we can say that a set of variables, $\mathcal{Z}$, satisfies the front-door criterion, given the graph, G , and a pair of variables, $X \rightarrow \dots \rightarrow Y$, if the following applies (Pearl et al., 2016):

- $\mathcal{Z}$ intercepts all directed paths from X to Y
- There are no open back-door paths from X to $\mathcal{Z}$
- All back-door paths from $\mathcal{Z}$ to Y are blocked by X

Front-door in practice

Let's implement a hypothetical model of our GPS example. You can find the code for this chapter in the following notebook: <https://bit.ly/causal-ntbk-06>.

First, let's define a **structural causal model (SCM)** that will generate hypothetical data for us. We're going to implement it as a Python class, similar to what we did in *Chapter 2*. Let's start with the imports:

```
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
plt.style.use('fivethirtyeight')
```

We import basic scientific packages. Note that this time we did not import Statsmodels' (Seabold and Perktold, 2010) linear regression module but, rather, the implementation from Scikit-learn (Pedregosa et al., 2011).

We did this on purpose to leverage the simple and intuitive interface that Scikit-learn offers. At the same time, we'll be less interested in the well-formatted output of the model results – a great feature of Statsmodels.

Perfect! Let's define our SCM!

```
class GPSTMemorySCM:

    def __init__(self, random_seed=None):
        self.random_seed = random_seed
        self.u_x = stats.truncnorm(0, np.infty, scale=5)
        self.u_y = stats.norm(scale=2)
        self.u_z = stats.norm(scale=2)
        self.u = stats.truncnorm(0, np.infty, scale=4)
```

In the `.init()` method, we define the distributions for all the exogenous variables in the model (we omitted them for readability in the preceding figures). We use a truncated normal distribution for `u_x` and `u` to restrict them to positive values and normal distribution for `u_y` and `u_z`

For clarity, you might want to take a look at *Figure 6.12*, which shows our full graphical model with exogenous variables:

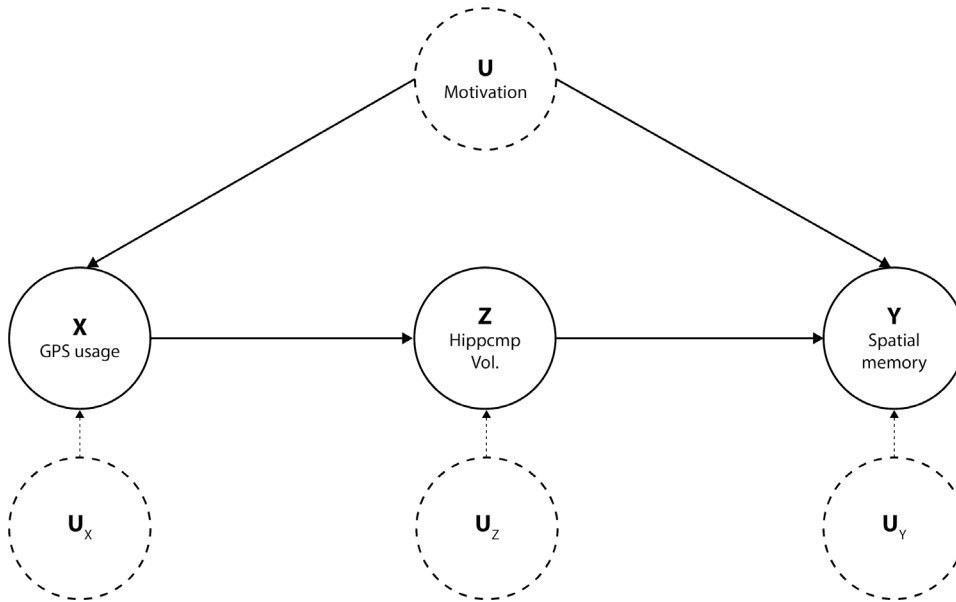


Figure 6.12 – A full model with exogenous variables

Next, we define the `.sample()` method.

Calling this method will return an *observational* distribution from our system. Note that all the coefficients are hypothetical and not based on actual research:

```
def sample(self, sample_size=100, treatment_value=None):
    """Samples from the SCM"""
    if self.random_seed:
        np.random.seed(self.random_seed)

    u_x = self.u_x.rvs(sample_size)
    u_y = self.u_y.rvs(sample_size)
    u_z = self.u_z.rvs(sample_size)
    u = self.u.rvs(sample_size)

    if treatment_value:
        gps = np.array([treatment_value]*sample_size)
    else:
        gps = u_x + 0.7*u

    hippocampus = -0.6*gps + 0.25*u_z
    memory = 0.7*hippocampus + 0.25*u

    return gps, hippocampus, memory
```

First, we fix the random seed if a user provided a value for it.

Next, we sample exogenous variables.

Finally, we compute the values of the three observed variables in our model, `gps`, `hippocampus`, and `memory`, which represent GPS usage, hippocampal volume, and spatial memory change respectively.

You might have noticed that there's an additional `if` statement that checks for `treatment_value`. It allows us to generate *interventional* distribution from the model if a value for `treatment_value` is provided.

The last method in our SCM implementation is `.intervene()`. It's a syntactic sugar wrapper around `.sample()`. The `.intervene()` method returns an interventional distribution from our model:

```
def intervene(self, treatment_value, sample_size=100):
    """Intervenes on the SCM"""
    return self.sample(treatment_value=treatment_value, sample_
size=sample_size)
```

Its purpose is to make the code cleaner and our process more explicit.

Note that passing either `None` or `0` as a treatment value will result in a special case of *null* intervention, and the outcome will be identical to the observational sample.

Great! Let's instantiate the model and generate some observational data:

```
scm = GPSTMemorySCM()
gps_obs, hippocampus_obs, memory_obs = scm.sample(600)
```

Generating observational data is as simple as calling the `.sample()` method.

Next, let's run an experiment. We will use a range of treatments from 1 to 20 units of GPS usage:

```
treatments = []
experiment_results = []

# Sample over a range of treatments
for treatment in np.arange(1, 21):
    gps_hours, hippocampus, memory = scm.intervene(treatment_
value=treatment, sample_size=30)
    experiment_results.append(memory)
    treatments.append(gps_hours)
```

For each treatment value, we sample 30 observations. We store treatment values and outcome values in the `treatments` and `experiment_results` lists respectively.

Figure 6.13 presents the relationship between GPS usage and spatial memory change in observational and interventional samples:

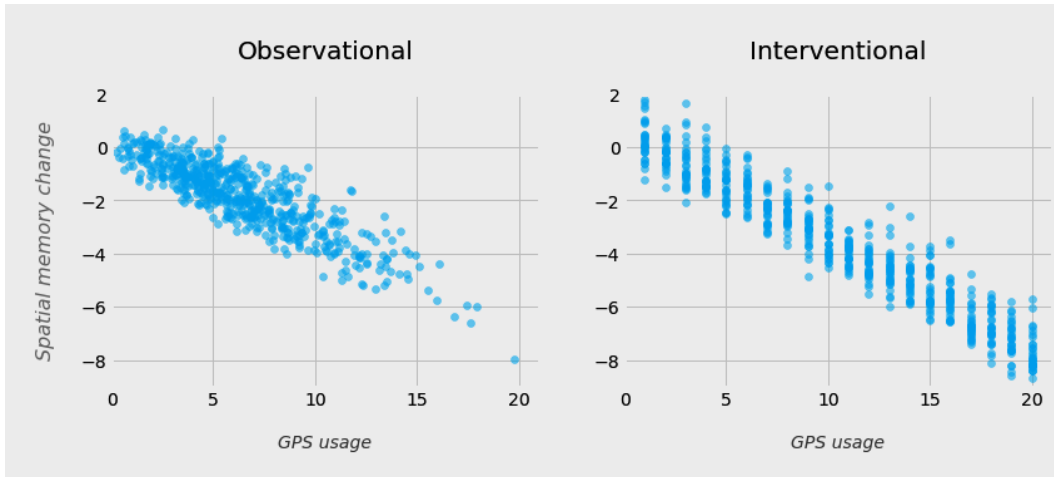


Figure 6.13 – Scatterplots of GPS usage versus spatial memory change

As you can see, the distributions in the scatterplots differ.

Let's fit two linear regression models – one on the observational data and one on the interventional data – and compare the results. What results do you expect?

Let's check your guess!

First, let's instantiate two regression models and train them:

```
lr_naive = LinearRegression()
lr_naive.fit(
    X=gps_obs.reshape(-1, 1),
    y=memory_obs
)
```

We train the first model, `lr_naive`, on the observational data. We regress spatial memory change on GPS usage.

You might have noticed that when we pass `gps_obs` to the `.fit()` method, we change the array's shape. This is because `sklearn` models require 2D arrays of shape (n, d) , where n is the number of observations and d is the dimensionality of the design matrix (or a number of features if you will), and our original array was 1D with shape $(n,)$ as we only have one feature.

The second model will use the same variables but generated in our experiment, rather than recorded from observations.

Before we train it, we need to unpack our treatment (GPS usage) and outcome (spatial memory change). This is because we generated 30 samples for each of the 20 intervention levels, and we stored them as a nested list of lists.

For ease of comparison with the observational models, we want to reshape the model to have 600 observations rather than 20*30 observations:

```
treatments_unpack = np.array(treatments).flatten()
results_unpack = np.array(experiment_results).flatten()

lr_experiment = LinearRegression()
lr_experiment.fit(
    X=treatments_unpack.reshape(-1, 1),
    y=results_unpack
)
```

Perfect. Now, let's generate predictions from both models on test data:

```
X_test = np.arange(1, 21).reshape(-1, 1)

preds_naive = lr_naive.predict(X_test)
preds_experiment = lr_experiment.predict(X_test)
```

We start by generating test data. It's as simple as generating a sequence between 1 and 20. These numbers quantify the number of units of GPS usage.

Next, we query both models to generate predictions on test data. *Figure 6.14* presents a scatterplot of the interventional distribution and predictions from both models:

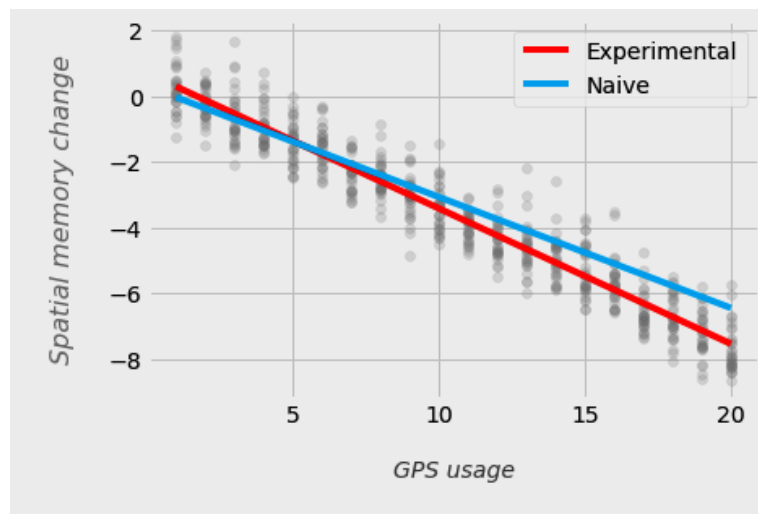


Figure 6.14 – A scatterplot of the interventional distribution and two fitted regression lines

We can see pretty clearly that the naive model does not fit the experimental data very well.

Let's compare the values for the regression coefficients for both models:

```
print(f'Naive model:\n{lr_naive.coef_}\n')
print(f'Experimental model:\n{lr_experiment.coef_}')
```

This results in the following output:

```
Naive model:
-0.3246130171160016

Experimental model:
-0.4207238110947806
```

As expected, the coefficient values for both models differ.

Let's figure out how to get a valid causal coefficient from observational data using the front-door criterion in three simple steps.

The linear bridge to the causal promised land

It turns out that when we're lucky enough that our model of interest is linear and the front-door criterion can be applied, we can compute the valid estimate of the causal effect of X on Y in three simple steps:

- Fit a model, $Z \sim X$
- Fit a model, $Y \sim Z + X$
- Multiply the coefficients from model 1 and model 2

Let's code it:

1. First, we train the model to regress Z on X ($Z \sim X$). Note that we only use observational data to fit this (and the following) model:

```
lr_zx = LinearRegression()
lr_zx.fit(
    X=gps_obs.reshape(-1, 1),
    y=hippocampus_obs
)
```

2. Next, let's train the model to regress Y on X and Z . Note that in both cases, we follow the same logic that we followed in the continuous case described previously:

```
lr_yxz = LinearRegression()
lr_yxz.fit(
    X=np.array([gps_obs, hippocampus_obs]).T,
    y=memory_obs
)
```

3. Finally, let's multiply the coefficients for both models:

```
lr_zx.coef_[0] * lr_yxz.coef_[1]
```

We take the 0th coefficient from the first model (there's just one coefficient, for GPS usage) and the 1st coefficient for the second model (because we're interested in the effect of hippocampal volume on spatial memory *given* GPU usage), and we multiply them together.

This gives us the following estimate of the causal effect:

```
-0.43713599902679
```

Good job! This is pretty close to the experimental model!

Values estimated from experiments and values estimated from observational data may differ in finite sample regimes, and that's utterly natural. The larger the sample size, the smaller the discrepancy between them we should expect *on average*.

Great!

We saw that the front-door-adjusted estimate was pretty close to the estimate obtained from the experimental data, but what actually is the true effect that we're trying to estimate, and how close are we?

We can answer this question pretty easily if we have a full linear SCM. The true effect in a model such as ours is equal to the product of coefficients on causal paths from $X \rightarrow Z$ and $Z \rightarrow Y$. The idea of multiplying the coefficients on a directed causal path can be traced back to Sewall Wright's *path analysis*, introduced as early as 1920 (Wright, 1920).

In our case, the true causal effect of GPS usage on spatial memory will be $-0.6 * 0.7 = -0.42$. The coefficients (-0.6 and 0.7) can be read from the definition of our SCM.

It turns out that our front-door-adjusted and experimental estimates were pretty close (~4% and <1% errors respectively), while the naïve estimate was more than 22% off. Imagine that you can improve the conversion rate of a marketing campaign by 20% for your client – an actual result that one of the causal machine learning companies where a colleague of mine used to work demonstrated.

It's time to conclude this section. We learned what the front-door criterion is. We discussed three conditions (and one additional assumption) necessary to make the criterion work and showed how to derive an *adjustment formula* from the basic principles. Finally, we built an SCM and generated observational and interventional distributions to show how the front-door criterion can be used to accurately approximate experimental results from observational data.

Are there other criteria out there? Let's do-calculus!

In the real world, not all causal graphs will have a structure that allows the use of the back-door or front-door criteria. Does this mean that we cannot do anything about them?

Fortunately, no. Back-door and front-door criteria are special cases of a more general framework called *do*-calculus (Pearl, 2009). Moreover, *do*-calculus has been proven to be complete (Shpitser and Pearl, 2006), meaning that if there is an identifiable causal effect in a given DAG, G , it can be found using the rules of *do*-calculus.

What are these rules?

The three rules of do-calculus

Before we can answer the question, we need to define some new helpful notation.

Given a DAG G , we can say that $G_{\bar{X}}$ is a modification of G , where we removed all the *incoming* edges to the node X . We will call G_x a modification of G , where we removed all the *outgoing* edges from the node X .

For example, $G_{\bar{X}Z}$ will denote a DAG, G , where we removed all the incoming edges to the node X and all the outgoing edges from the node Z .

Perfect. Now, let's see the rules (Pearl, 2009, and Malina, 2020):

- *Rule 1:* When an observation can be ignored:

$$P(Y = y | do(X = x), Z = z, W = w) = P(Y = y | do(X = x), W = w) \text{ if } (Y \perp\!\!\!\perp Z | X, W)_{G_x}$$

- *Rule 2:* When intervention can be treated as an observation:

$$P(Y = y | do(X = x), do(Z = z), W = w) = P(Y = y | do(X = x), Z = z, W = w) \text{ if } (Y \perp\!\!\!\perp Z | X, W)_{G_{\bar{X}Z}}$$

- *Rule 3:* When intervention can be ignored:

$$P(Y = y | do(X = x), do(Z = z), W = w) = P(Y = y | do(X = x), W = w) \text{ if } (Y \perp\!\!\!\perp Z | X, W)_{G_{\bar{X}, Z(W)}}$$

In *rule 3*, $Z(W)$ is the set of Z -nodes that are not ancestors of any W -nodes in the altered DAG, $G_{\bar{X}}$.

These rules might look pretty overwhelming! Let's try to decode their meaning.

Rule 1 tells us that we can *ignore* any observational (set of) variable(s), Z , when Z and the outcome, Y , are independent, given X and W in a modified DAG, G_x .

Rule 2 tells us that any intervention over a (set of) variable(s), Z , can be *treated as an observation* when Z and the outcome Y are independent given X and W in a modified DAG $G_{\bar{X}Z}$.

Finally, *rule 3* tells us that any intervention over a (set of) variable(s), Z , can be *ignored* when Z and the outcome, Y , are independent, given X and W in a modified DAG, $G_{\bar{X}, Z(W)}$.

All this might sound complicated at first, until you realize that what it requires in practice is to take your DAG, find the (set of) confounders (denoted as Z in our rules), and check whether any of the rules apply. Plus, you can stack the transformations into arbitrary long sequences if this helps! The good part is that this work can also be automated.

It might take some time to fully digest the rules of *do*-calculus, and that's OK. Once you get familiar with them, you'll have a very powerful tool in your toolbox.

If you want to learn more about *do*-calculus, check out Pearl (2009) for formal definitions and step-by-step examples, Spitzer and Pearl (2006) for the proof of completeness, and Stephen Malina's blog for intuitive understanding (Malina, 2020 – <https://stephenmalina.com/post/2020-03-09-front-door-do-calc-derivation/>).

Before we conclude this section, let's see one more popular method that can be used to identify causal effects.

Instrumental variables

Instrumental variables (IVs) are a family of deconfounding techniques that are hugely popular in econometrics. Let's take a look at the DAG in Figure 6.15:

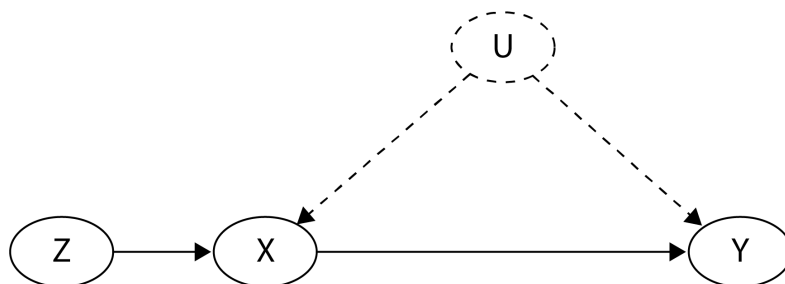


Figure 6.15 – An example DAG for the IV technique

We're interested in estimating the causal effect of X on Y . You can see that we cannot use the back-door criterion here because U is unobserved. We cannot use the front-door criterion either because there's no mediator between X and Y .

Not all is lost though! It turns out that we can use the IV technique to estimate our causal effect of interest. Let's see how to do it.

The three conditions of IVs

Instrumental variable methods require a special variable called an *instrument* to be present in a graph. We will use Z to denote the instrument. Our effect of interest is the causal effect of X on Y .

An *instrument* needs to meet the following three conditions (Hernán and Robins, 2020):

- The instrument, Z , is associated with X
- The instrument, Z , doesn't affect Y in any way except through X
- There are no common causes of Z and Y

The first condition talks about *association* rather than *causation*. The nature of the relationship between Z and X determines how much information we'll be able to extract from our instrument. Theoretically speaking, we can even use instruments that are only weakly (non-directly) associated with X (in such a case, they are called **proxy instruments**), yet there's a cost to this.

In certain cases, the only thing we'll be able to obtain will be the lower and upper bounds of the effect, and in some cases, these bounds might be very broad and, therefore, not very useful (Hernán and Robins, 2020).

We're lucky though!

Calculating causal effects with IVs

Let's take a look at *Figure 6.15* once again. The variable Z is associated with X , it does not affect Y other than through X , and there are no common causes of Z and Y ; therefore, Z meets all of the criteria of being an *instrument*. Moreover, in our DAG, the relationship between Z and X is causal and direct, which allows us to approximate the exact causal effect (as opposed to just bounds)!

To calculate the causal effect of X on Y in a linear case, all we need to do is fit two linear regression models and compute the ratio of their coefficients!

The two models are as follows:

- $Y \sim Z$
- $Y \sim X$

We compute the ratio by dividing the coefficient for the first model by the coefficient from the second model.

Et voilà!

To see a step-by-step example of computing the IV estimation in a linear model, check the notebook accompanying this chapter (<https://bit.ly/causal-ntbk-06>).

Estimation using IVs can be extended to non-linear and non-parametric cases (for example, Li et al., 2022, and Carroll et al., 2004).

All this makes IVs pretty flexible and broadly adopted, yet in practice, it might be difficult to find good instruments or verify that the necessary assumptions (for instance, a lack of influence of Z on Y other than through X) are met.

There are a couple of resources if you want to dive deeper into IVs. For a great technical and formal overview, check out *Chapter 16* of the excellent book *Causal Inference: What If?* by Miguel Hernán and James Robins from the Harvard School of Public Health (Hernán and Robins, 2020).

For practical advice on how to find good instruments, intuitions, and great real-world examples, check out *Chapter 7* of Scott Cunningham’s *Causal Inference: The Mixtape* (Cunningham, 2021). I am sure you’ll love the latter, in particular if you’re a hip-hop aficionado.

In this section, we learned about *do*-calculus, a flexible framework to identify causal effects that generalizes back-door and front-door criteria. We discussed the three rules of *do*-calculus that allow us to find causal effects in any DAG that is identifiable (hence, the completeness of *do*-calculus).

In the second part of this section, we introduced the IV technique – a popular method of causal effect identification and estimation that has been widely adopted in econometrics, epidemiology, and social sciences.

Finally, we learned how to use IVs in a linear case and linked to the resources demonstrating how the method can be extended to non-linear and non-parametric cases.

Wrapping it up

We learned a lot in this chapter, and you deserve some serious applause for coming this far!

In this chapter, we learned a lot. We started with the notion of *d*-separation. Then, we showed how *d*-separation is linked to the idea of an *estimand*. We discussed what causal estimands are and what their role is in the causal inference process.

Next, we discussed two powerful methods of causal effect identification, the back-door and front-door criteria, and applied them to our ice cream and GPS usage examples.

Finally, we presented a generalization of front-door and back-door criteria, the powerful framework of *do*-calculus, and introduced a family of methods called instrumental variables, which can help us identify causal effects where other methods fail.

The set of methods we learned in this chapter gives us a powerful causal toolbox that we can apply to real-world problems.

In the next chapter, we’ll demonstrate how to properly structure an end-to-end causal inference process using the DoWhy library (Sharma and Kiciman, 2020), and we will get ready to set sail for the journey into the stormy waters of causal machine learning.

Answer

Controlling for *B* (*Figure 6.7*) essentially removes *A*’s influence on *X* and *Y*. If we remove *A* from the graph, it will not change anything (up to noise) in our estimate of the relationship strength between *X* and *Y*. Note that in a graph with a removed node *A*, controlling for *B* becomes irrelevant (it does not hurt us to do so, but there’s no benefit to it either).